

Diseño de Arquitectura de Solución

BP Internet Banking Platform

Arquitecto de Soluciones: **Juan Pablo Tandazo Rodríguez**

Fecha: **Junio 2026**

Tabla de Contenido

1. Introducción
2. Objetivos de la Solución
3. Requerimientos del Negocio
4. Decisiones Arquitectónicas
5. Arquitectura C4
 - Context Diagram
 - Container Diagram
 - Component Diagram
6. Arquitectura de Autenticación OAuth2
7. Arquitectura de Onboarding Digital
8. Arquitectura Event Driven
9. Arquitectura AWS
10. Alta Disponibilidad y Disaster Recovery
11. Seguridad
12. Cumplimiento Normativo
13. Patrones Arquitectónicos
14. Costos y Escalabilidad
15. Conclusiones

1. Introducción

Contexto

BP requiere una plataforma de banca digital que permita:

- Consulta de productos
- Consulta de movimientos
- Transferencias internas
- Transferencias interbancarias
- Pagos
- Onboarding digital
- Notificaciones regulatorias

La solución debe garantizar:

- Alta disponibilidad
 - Seguridad
 - Escalabilidad
 - Trazabilidad
 - Recuperación ante desastres
-

2. Objetivos de la Solución

Funcionales

- Consulta de movimientos históricos
- Consulta de información del cliente
- Transferencias internas
- Transferencias ACH
- Pagos
- Notificaciones

No Funcionales

- Disponibilidad > 99.95%
 - RPO ≤ 5 minutos
 - RTO ≤ 30 minutos
 - MFA
 - Escalabilidad horizontal
 - Auditoría completa
-

3. Requerimientos del Negocio

Requisito	Solución
Consulta movimientos	Transaction Service
Transferencias	Transfer Service
Datos cliente	Customer Service
Onboarding	Onboarding Service
OAuth	Identity Provider
Notificaciones	Notification Service
Auditoría	Audit Service
Cache frecuentes	Redis

4. Decisiones Arquitectónicas

Microservicios

Decisión

Arquitectura basada en microservicios.

Justificación 1

Permite escalamiento independiente.

Justificación 2

Reduce acoplamiento.

Alternativas evaluadas

- Monolito
 - SOA
-

Kafka

Decisión

Arquitectura orientada a eventos.

Justificación 1

Desacoplamiento.

Justificación 2

Alta resiliencia.

Alternativas

- RabbitMQ
 - SQS
-

PostgreSQL Aurora

Justificación 1

Alta disponibilidad.

Justificación 2

Compatibilidad SQL.

Redis

Justificación 1

Baja latencia.

Justificación 2

Cache Aside Pattern.

Flutter

Alternativas evaluadas

- Flutter
- React Native

Selección

Flutter

Justificación 1

Mayor rendimiento.

Justificación 2

Mejor soporte biométrico.

Angular SPA

Alternativas

- Angular
- React

Selección

Angular

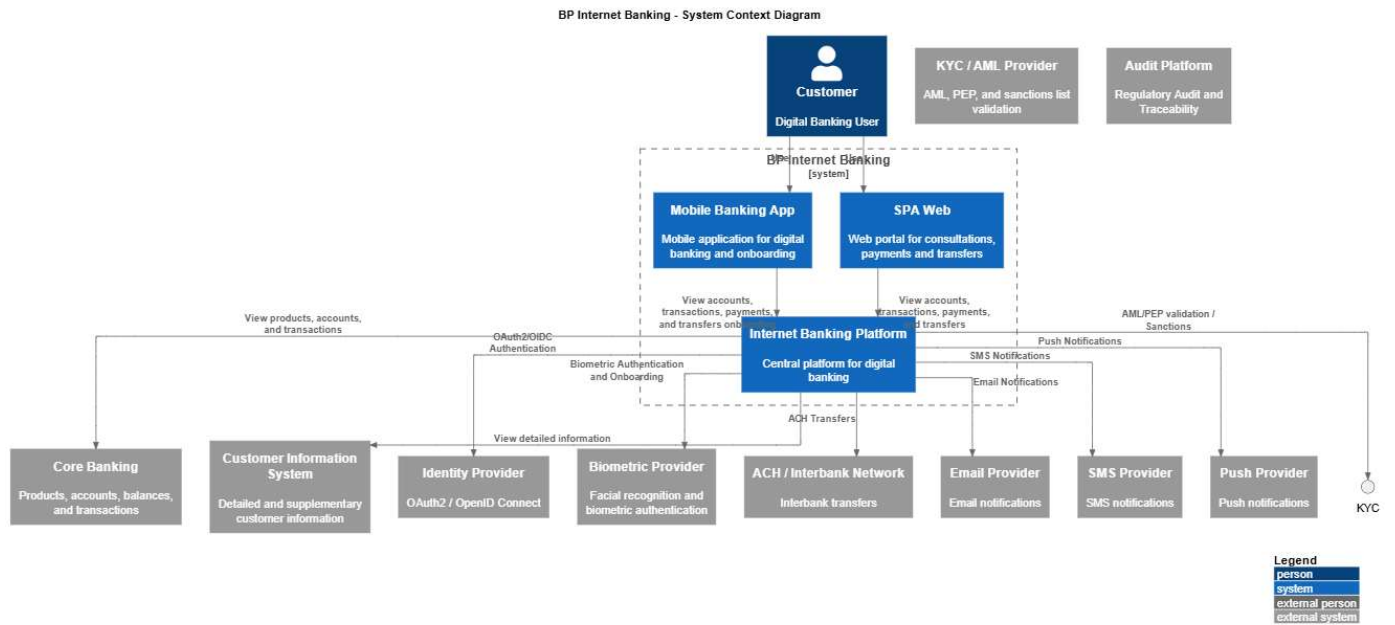
Justificación

- Arquitectura empresarial

- Tipado fuerte

5. Arquitectura C4

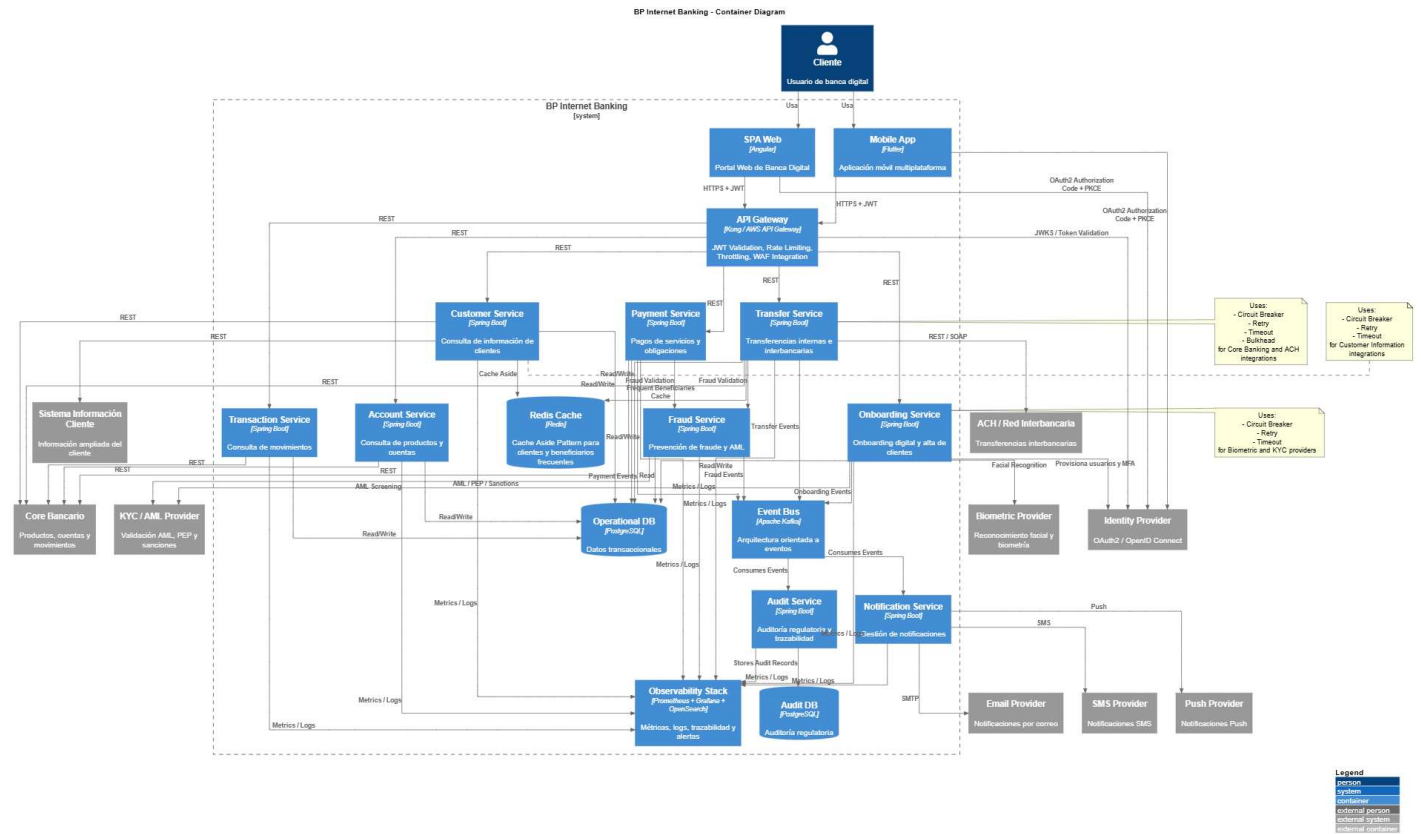
5.1 Context Diagram



Explicación

Mostrar actores y sistemas externos.

5.2 Container Diagram

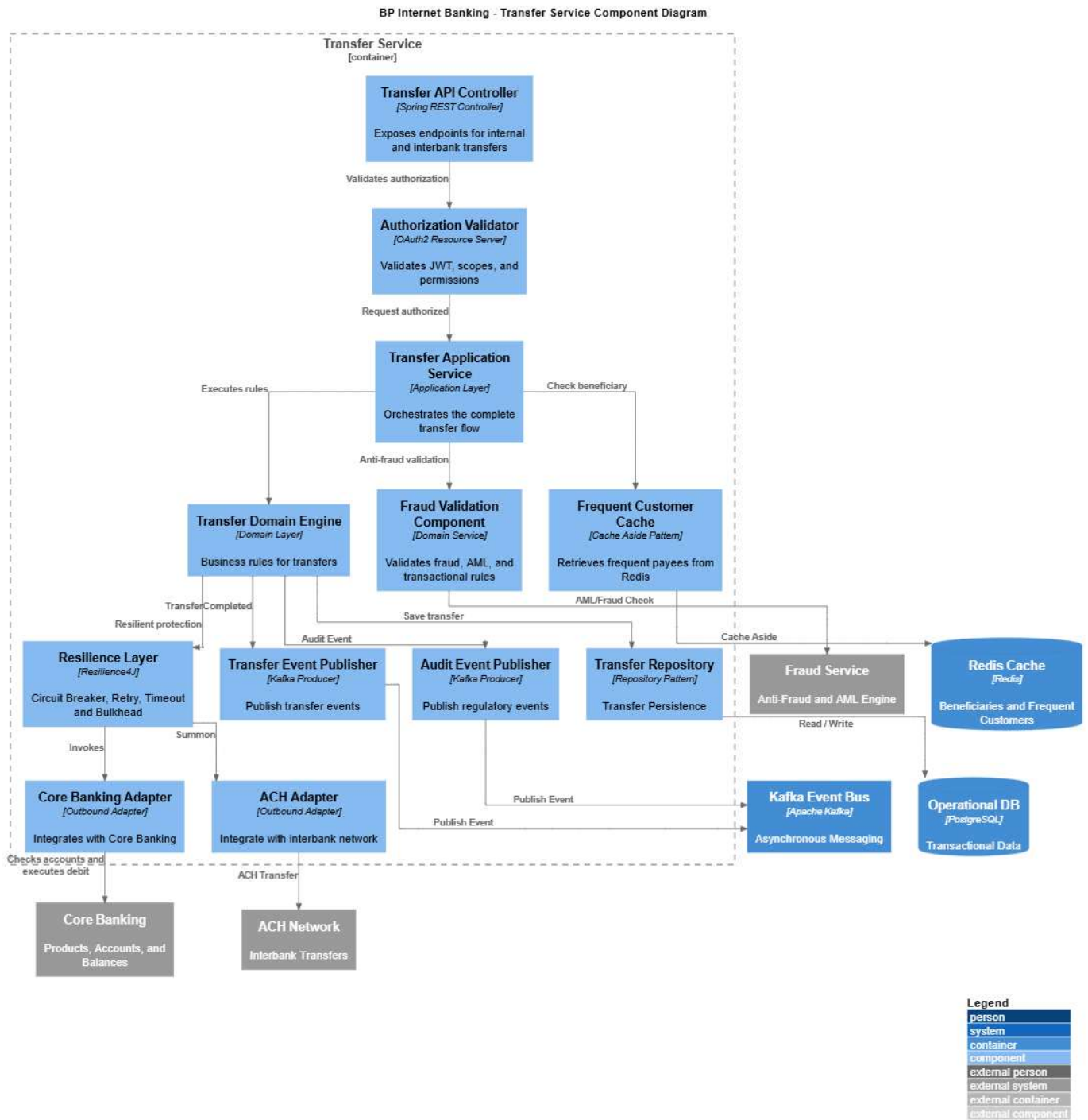


Explicación

Describe:

- SPA
- Mobile
- API Gateway
- Microservicios
- Redis
- Kafka
- PostgreSQL

5.3 Component Diagram



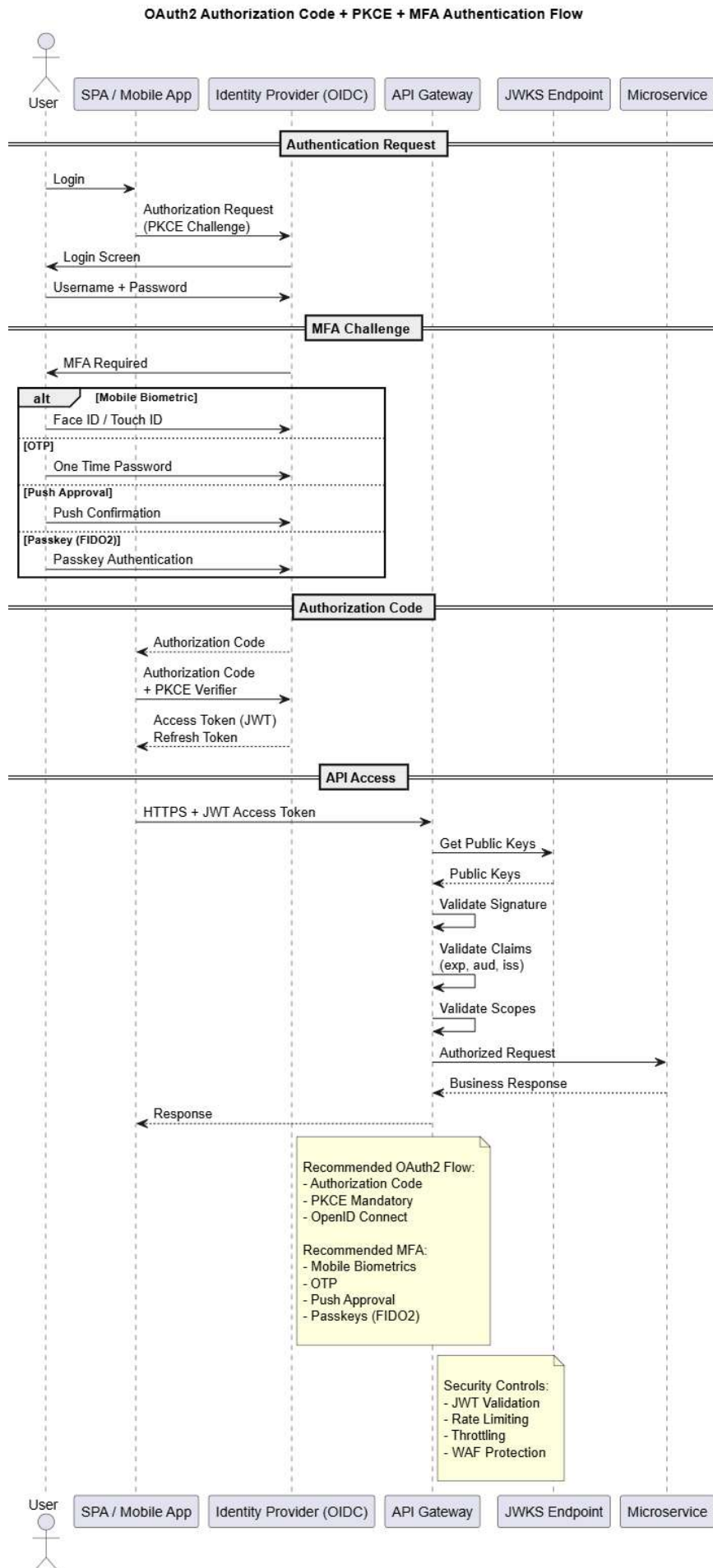
Explicación

Transfer Service:

- Controller
- Domain
- Repository
- Kafka Producer
- Cache Aside

- Circuit Breaker
-

6. Arquitectura OAuth2



Flujo seleccionado

OAuth2 Authorization Code + PKCE

Justificación 1

Recomendado para SPA.

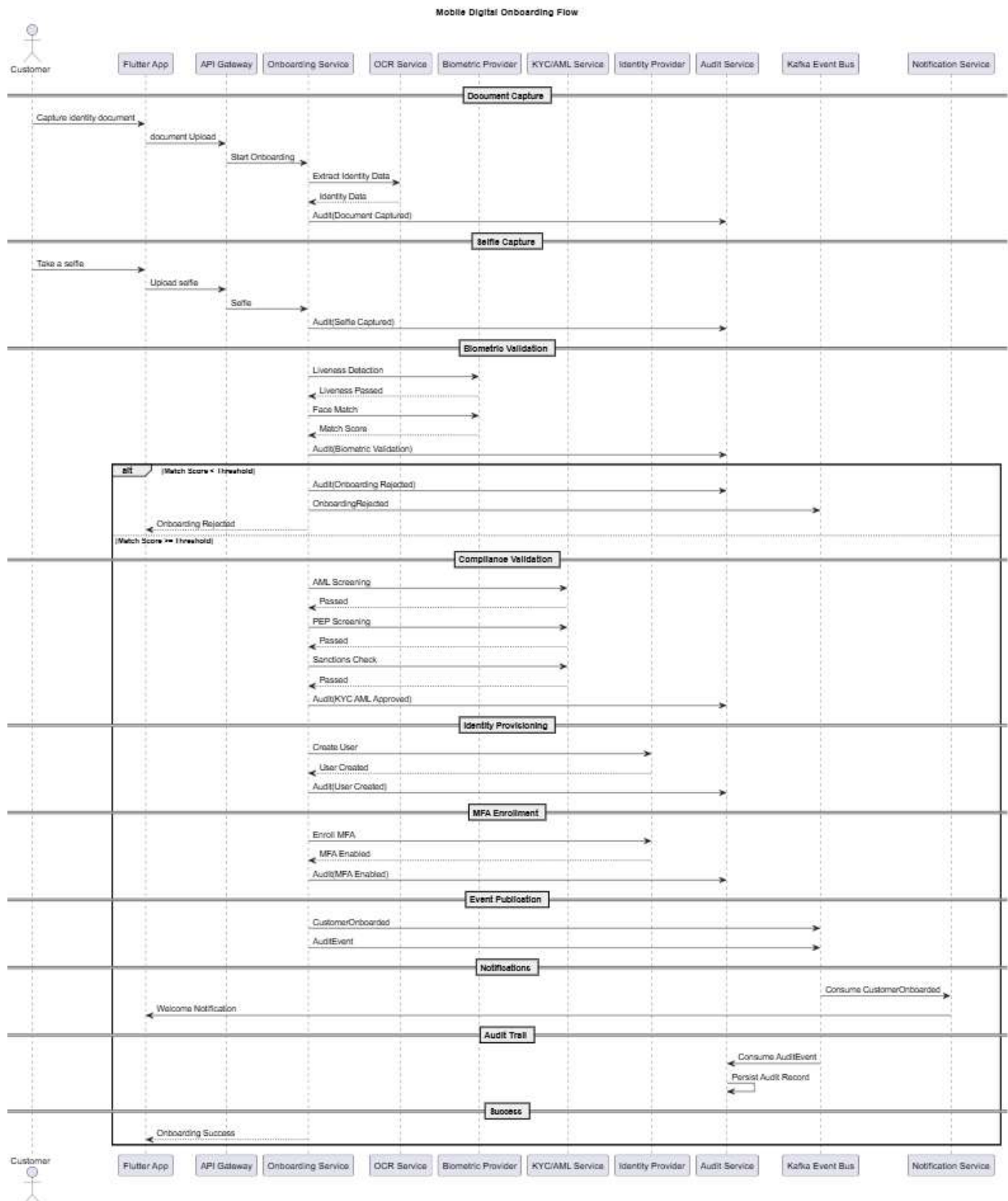
Justificación 2

Recomendado para Mobile.

MFA

- OTP
 - Push
 - Face ID
 - Passkeys FIDO2
-

7. Arquitectura de Onboarding

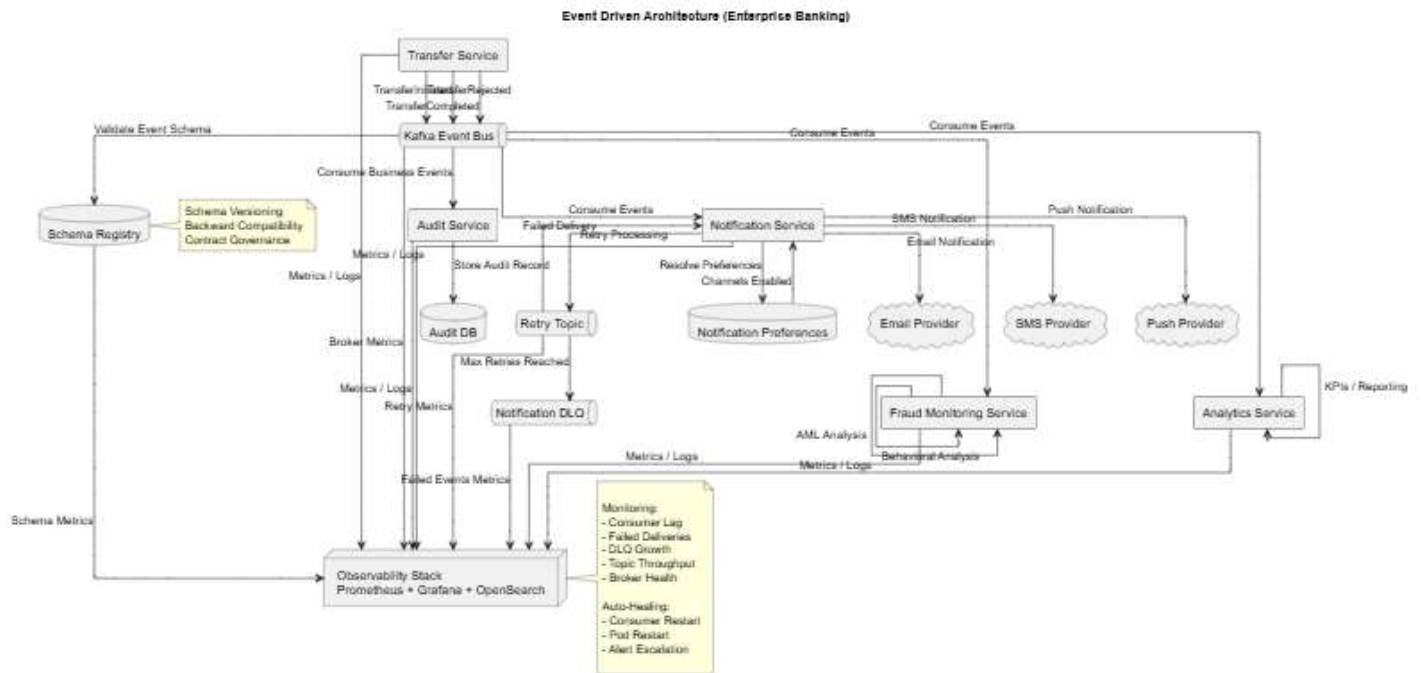


Proceso

1. Captura documento
2. OCR
3. Selfie
4. Liveness

5. Face Match
 6. AML
 7. PEP
 8. Provisionamiento
 9. MFA
-

8. Arquitectura Event Driven



Eventos

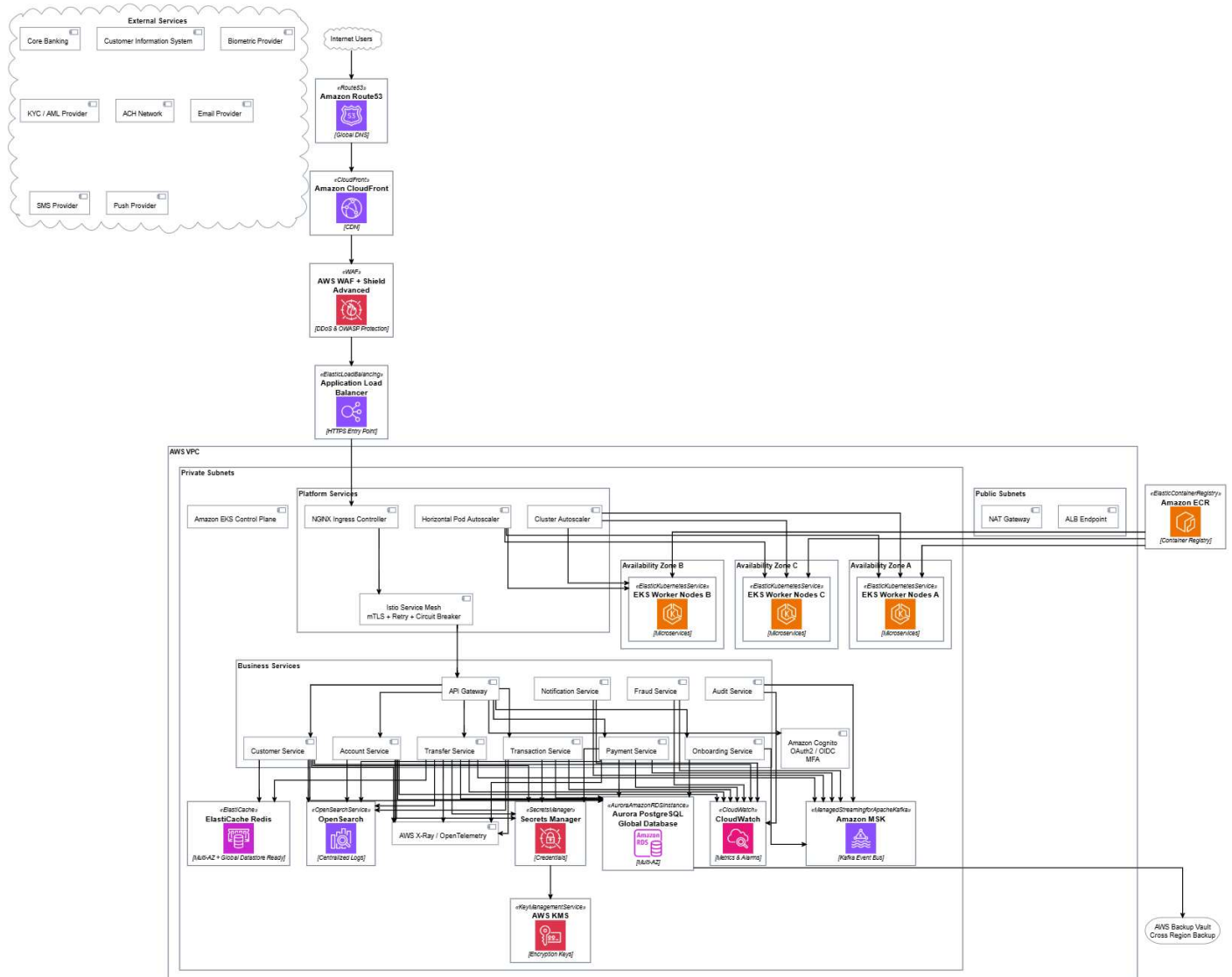
- TransferCompleted
- TransferRejected
- CustomerOnboarded

Beneficios

- Desacoplamiento
- Escalabilidad
- Auditoría

9. Arquitectura AWS

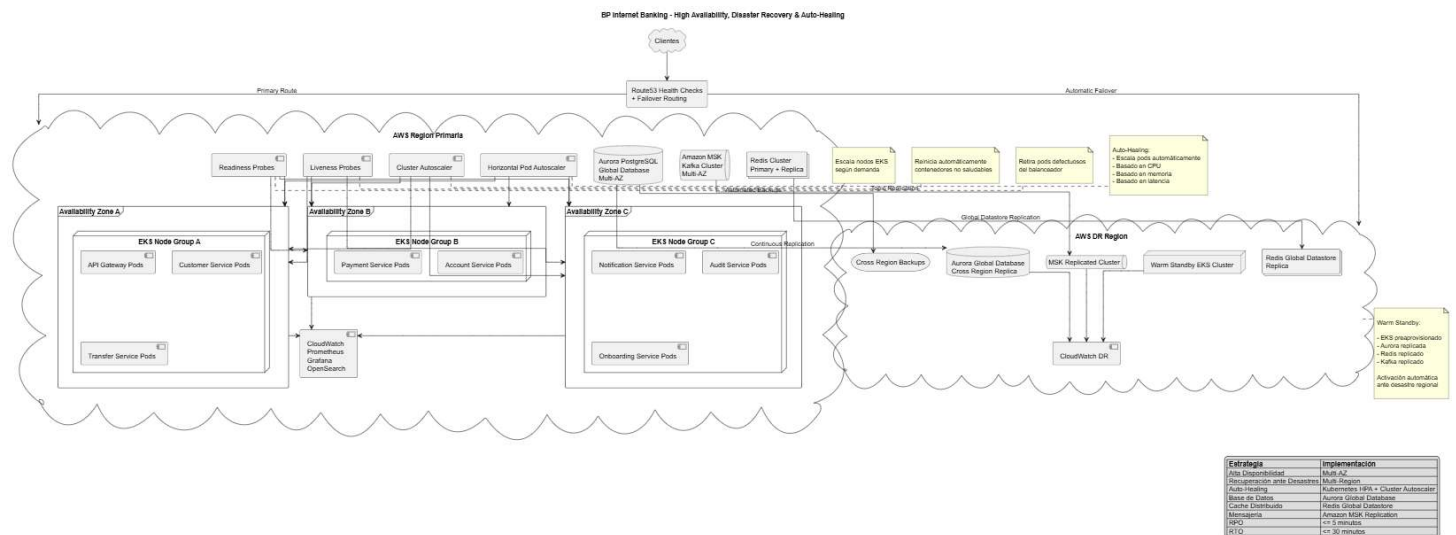
BP Internet Banking - AWS Production Architecture



Servicios AWS

Servicio	Uso
EKS	Microservicios
Aurora	Datos
Redis	Cache
MSK	Kafka
Cognito	OAuth
CloudWatch	Monitoreo
OpenSearch	Logs
Route53	DNS
WAF	Seguridad

10. Alta Disponibilidad y DR



Estrategia

HA

- Multi AZ

DR

- Multi Region

Auto Healing

- HPA
- Cluster Autoscaler
- Liveness
- Readiness

RPO

<= 5 minutos

RTO

<= 30 minutos

11. Seguridad

OAuth2 + OIDC

Autenticación.

MFA

Segundo factor.

TLS 1.3

Comunicación segura.

WAF

Protección OWASP.

KMS

Cifrado.

Secrets Manager

Credenciales.

mTLS

Comunicación interna.

12. Cumplimiento Normativo

PCI DSS

Protección de datos financieros.

ISO 27001

Gestión de seguridad.

OWASP ASVS

Aplicaciones seguras.

NIST CSF

Marco de ciberseguridad.

Ley de Protección de Datos Personales (Ecuador)

Cumplimiento regulatorio.

AML

Prevención de lavado.

PEP

Personas expuestas políticamente.

13. Patrones Arquitectónicos

Patrón	Uso
Microservices	Desacoplamiento
API Gateway	Entrada única
Event Driven	Asincronía
Cache Aside	Clientes frecuentes
Repository	Persistencia

Patrón	Uso
Adapter	Core y ACH
Circuit Breaker	Resiliencia
Retry	Recuperación
Bulkhead	Aislamiento
CQRS (opcional)	Lecturas futuras

14. Costos y Escalabilidad

Escalabilidad

Horizontal mediante:

- Kubernetes
- HPA
- Autoscaler

Optimización de Costos

- Redis para reducir llamadas al Core
 - Kafka para desacoplamiento
 - CloudFront para contenido estático
-

15. Conclusiones

La solución propuesta proporciona:

- Arquitectura desacoplada
- Alta disponibilidad
- Recuperación ante desastres
- Seguridad bancaria
- Escalabilidad
- Auditoría regulatoria
- Preparación para crecimiento futuro

CODIGO

```
@startuml C4_Context
```

```
top to bottom direction
```

```
skinparam linetype ortho
```

```
!include ../C4-PlantUML/C4_Context.puml
```

```
title BP Internet Banking - System Context Diagram
```

```
LAYOUT_TOP_DOWN()
```

```
Person(customer, "Customer", "Digital Banking User")
```

```
System_Boundary(bp, "BP Internet Banking") {
```

```
    System(webApp, "SPA Web", "Web portal for consultations, payments and transfers")
```

```
    System(mobileApp, "Mobile Banking App", "Mobile application for digital banking and onboarding")
```

```
    System(internetBanking, "Internet Banking Platform", "Central platform for digital banking")
```

```
}
```

```
System_Ext(core, "Core Banking", "Products, accounts, balances, and transactions")
```

```
System_Ext(customerInfo, "Customer Information System", "Detailed and supplementary customer information")
```

```
System_Ext(idp, "Identity Provider", "OAuth2 / OpenID Connect")
```

```
System_Ext(biometric, "Biometric Provider", "Facial recognition and biometric authentication")
```

```
System_Ext(kyc, "KYC / AML Provider", "AML, PEP, and sanctions list validation")
```

```
System_Ext(interbank, "ACH / Interbank Network", "Interbank transfers")
```

```
System_Ext(emailProvider, "Email Provider", "Email notifications")
```

```
System_Ext(smsProvider, "SMS Provider", "SMS notifications")
```

System_Ext(pushProvider, "Push Provider", "Push notifications")

System_Ext(auditPlatform, "Audit Platform", "Regulatory Audit and Traceability")

'=====

' CLIENT CHANNELS

'=====

Rel(customer, webApp, "Use")

Rel(customer, mobileApp, "Use")

'=====

' DIGITAL BANKING

'=====

Rel(webApp, internetBanking, "View accounts, transactions, payments, and transfers")

Rel(mobileApp, internetBanking, "View accounts, transactions, payments, and transfers onboarding")

'=====

' AUTHENTICATION

'=====

Rel(internetBanking, idp, "OAuth2/OIDC Authentication")

Rel(internetBanking, biometric, "Biometric Authentication and Onboarding")

'=====

' CUSTOMER DATA

'=====

Rel(internetBanking, core, "View products, accounts, and transactions")

Rel(internetBanking, customerInfo, "View detailed information")

```
'=====
' ONBOARDING
'=====
```

Rel(internetBanking, KYC, "AML/PEP validation / Sanctions")

```
'=====
' TRANSFERS
'=====
```

Rel(internetBanking, interbank, "ACH Transfers")

```
'=====
' NOTIFICATIONS
'=====
```

Rel(internetBanking, emailProvider, "Email Notifications")

Rel(internetBanking, smsProvider, "SMS Notifications")

Rel(internetBanking, pushProvider, "Push Notifications")

LAYOUT_WITH_LEGEND()

@enduml

@startuml C4_Container

top to bottom direction

skinparam linetype ortho

!include ../C4-PlantUML/C4_Container.puml

title BP Internet Banking - Container Diagram

Person(customer, "Cliente", "Usuario de banca digital")

System_Ext(core, "Core Bancario", "Productos, cuentas y movimientos")

System_Ext(customerInfo, "Sistema Información Cliente", "Información ampliada del cliente")

System_Ext(idp, "Identity Provider", "OAuth2 / OpenID Connect")

System_Ext(faceProvider, "Biometric Provider", "Reconocimiento facial y biometría")

System_Ext(kycProvider, "KYC / AML Provider", "Validación AML, PEP y sanciones")

System_Ext(emailProvider, "Email Provider", "Notificaciones por correo")

System_Ext(smsProvider, "SMS Provider", "Notificaciones SMS")

System_Ext(pushProvider, "Push Provider", "Notificaciones Push")

System_Ext(interbank, "ACH / Red Interbancaria", "Transferencias interbancarias")

System_Boundary(bp, "BP Internet Banking") {

Container(web, "SPA Web", "Angular", "Portal Web de Banca Digital")

Container(mobile, "Mobile App", "Flutter", "Aplicación móvil multiplataforma")

Container(api, "API Gateway", "Kong / AWS API Gateway", "JWT Validation, Rate Limiting, Throttling, WAF Integration")

Container(customerSvc, "Customer Service", "Spring Boot", "Consulta de información de clientes")

Container(accountSvc, "Account Service", "Spring Boot", "Consulta de productos y cuentas")

Container(transactionSvc, "Transaction Service", "Spring Boot", "Consulta de movimientos")

Container(transferSvc, "Transfer Service", "Spring Boot", "Transferencias internas e interbancarias")

Container(paymentSvc, "Payment Service", "Spring Boot", "Pagos de servicios y obligaciones")

Container(onboardingSvc, "Onboarding Service", "Spring Boot", "Onboarding digital y alta de clientes")

Container(notificationSvc, "Notification Service", "Spring Boot", "Gestión de notificaciones")

Container(auditSvc, "Audit Service", "Spring Boot", "Auditoría regulatoria y trazabilidad")

Container(fraudSvc, "Fraud Service", "Spring Boot", "Prevención de fraude y AML")

ContainerDb(postgres, "Operational DB", "PostgreSQL", "Datos transaccionales")

ContainerDb(auditDb, "Audit DB", "PostgreSQL", "Auditoría regulatoria")

ContainerDb(redis, "Redis Cache", "Redis", "Cache Aside Pattern para clientes y beneficiarios frecuentes")

Container(kafka, "Event Bus", "Apache Kafka", "Arquitectura orientada a eventos")

Container(observability, "Observability Stack", "Prometheus + Grafana + OpenSearch", "Métricas, logs, trazabilidad y alertas")

}

'=====

' CLIENT CHANNELS

'=====

Rel(customer, web, "Usa")

Rel(customer, mobile, "Usa")

Rel(web, idp, "OAuth2 Authorization Code + PKCE")

Rel(mobile, idp, "OAuth2 Authorization Code + PKCE")

Rel(web, api, "HTTPS + JWT")

Rel(mobile, api, "HTTPS + JWT")

'=====

' TOKEN VALIDATION

'=====

Rel(api, idp, "JWKS / Token Validation")

'=====

' API ROUTING

'=====

Rel(api, customerSvc, "REST")

Rel(api, accountSvc, "REST")

Rel(api, transactionSvc, "REST")

Rel(api, transferSvc, "REST")

Rel(api, paymentSvc, "REST")

Rel(api, onboardingSvc, "REST")

'=====

' CORE SYSTEMS

'=====

Rel(customerSvc, core, "REST")

Rel(accountSvc, core, "REST")

Rel(transactionSvc, core, "REST")

Rel(transferSvc, core, "REST")

Rel(paymentSvc, core, "REST")

Rel(customerSvc, customerInfo, "REST")

'=====

' ONBOARDING

'=====

Rel(onboardingSvc, faceProvider, "Facial Recognition")

Rel(onboardingSvc, kycProvider, "AML / PEP / Sanctions")

Rel(onboardingSvc, idp, "Provisiona usuarios y MFA")

'=====

' FRAUD CONTROL

'=====

Rel(transferSvc, fraudSvc, "Fraud Validation")

Rel(paymentSvc, fraudSvc, "Fraud Validation")

Rel(fraudSvc, kycProvider, "AML Screening")

'=====

' DATABASE ACCESS

'=====

Rel(customerSvc, postgres, "Read/Write")

Rel(accountSvc, postgres, "Read/Write")

Rel(transactionSvc, postgres, "Read/Write")

Rel(transferSvc, postgres, "Read/Write")

Rel(paymentSvc, postgres, "Read/Write")

Rel(onboardingSvc, postgres, "Read/Write")

Rel(fraudSvc, postgres, "Read")

Rel(auditSvc, auditDb, "Stores Audit Records")

'=====

' CACHE ASIDE PATTERN

'=====

Rel(customerSvc, redis, "Cache Aside")

Rel(transferSvc, redis, "Frequent Beneficiaries Cache")

'=====

' INTERBANK TRANSFERS

'=====

Rel(transferSvc, interbank, "REST / SOAP")

'=====

' NOTIFICATIONS

'=====

Rel(notificationSvc, emailProvider, "SMTP")

Rel(notificationSvc, smsProvider, "SMS")

Rel(notificationSvc, pushProvider, "Push")

'=====

' EVENT DRIVEN

'=====

Rel(transferSvc, kafka, "Transfer Events")

Rel(paymentSvc, kafka, "Payment Events")

Rel(onboardingSvc, kafka, "Onboarding Events")

Rel(fraudSvc, kafka, "Fraud Events")

Rel(kafka, notificationSvc, "Consumes Events")

Rel(kafka, auditSvc, "Consumes Events")

'=====

' OBSERVABILITY

'=====

Rel(customerSvc, observability, "Metrics / Logs")

Rel(accountSvc, observability, "Metrics / Logs")

Rel(transactionSvc, observability, "Metrics / Logs")

Rel(transferSvc, observability, "Metrics / Logs")

Rel(paymentSvc, observability, "Metrics / Logs")

Rel(onboardingSvc, observability, "Metrics / Logs")

Rel(notificationSvc, observability, "Metrics / Logs")

Rel(auditSvc, observability, "Metrics / Logs")

Rel(fraudSvc, observability, "Metrics / Logs")

'=====

' RESILIENCE

'=====

note right of transferSvc

Uses:

- Circuit Breaker
- Retry
- Timeout
- Bulkhead

for Core Banking and ACH integrations

end note

note right of onboardingSvc

Uses:

- Circuit Breaker
- Retry
- Timeout

for Biometric and KYC providers

end note

note right of customerSvc

Uses:

- Circuit Breaker
- Retry
- Timeout

for Customer Information integrations

end note

LAYOUT_WITH_LEGEND()

@enduml

@startuml C4_Component

top to bottom direction

skinparam linetype ortho

!include ../C4-PlantUML/C4_Component.puml

title BP Internet Banking - Transfer Service Component Diagram

Container_Boundary(transferSvc, "Transfer Service") {

 Component(apiController, "Transfer API Controller", "Spring REST Controller", "Exposes endpoints for internal and interbank transfers")

 Component(authValidator, "Authorization Validator", "OAuth2 Resource Server", "Validates JWT, scopes, and permissions")

 Component(transferAppService, "Transfer Application Service", "Application Layer", "Orchestrates the complete transfer flow")

 Component(fraudValidator, "Fraud Validation Component", "Domain Service", "Validates fraud, AML, and transactional rules")

 Component(customerCache, "Frequent Customer Cache", "Cache Aside Pattern", "Retrieves frequent payees from Redis")

 Component(transferDomain, "Transfer Domain Engine", "Domain Layer", "Business rules for transfers")

 Component(coreAdapter, "Core Banking Adapter", "Outbound Adapter", "Integrates with Core Banking")

 Component(achAdapter, "ACH Adapter", "Outbound Adapter", "Integrate with interbank network")

 Component(eventPublisher, "Transfer Event Publisher", "Kafka Producer", "Publish transfer events")

 Component(auditPublisher, "Audit Event Publisher", "Kafka Producer", "Publish regulatory events")

Component(transferRepository, "Transfer Repository", "Repository Pattern", "Transfer Persistence")

Component(circuitBreaker, "Resilience Layer", "Resilience4J", "Circuit Breaker, Retry, Timeout and Bulkhead")

}

ContainerDb(postgres, "Operational DB", "PostgreSQL", "Transactional Data")

ContainerDb(redis, "Redis Cache", "Redis", "Beneficiaries and Frequent Customers")

Container(kafka, "Kafka Event Bus", "Apache Kafka", "Asynchronous Messaging")

System_Ext(core, "Core Banking", "Products, Accounts, and Balances")

System_Ext(interbank, "ACH Network", "Interbank Transfers")

System_Ext(fraudSvc, "Fraud Service", "Anti-Fraud and AML Engine")

' =====

' REQUEST FLOW

' =====

Rel(apiController, authValidator, "Validates authorization")

Rel(authValidator, transferAppService, "Request authorized")

Rel(transferAppService, fraudValidator, "Anti-fraud validation")

Rel(fraudValidator, fraudSvc, "AML/Fraud Check")

Rel(transferAppService, customerCache, "Check beneficiary")

Rel(customerCache, redis, "Cache Aside")

Rel(transferAppService, transferDomain, "Executes rules")

' =====

' PERSISTENCE

' =====

Rel(transferDomain, transferRepository, "Save transfer")

Rel(transferRepository, postgres, "Read / Write")

' =====

' CORE INTEGRATION

' =====

Rel(transferDomain, circuitBreaker, "Resilient protection")

Rel(circuitBreaker, coreAdapter, "Invokes")

Rel(coreAdapter, core, "Checks accounts and executes debit")

' =====

' ACH INTEGRATION

' =====

Rel(circuitBreaker, achAdapter, "Summon")

Rel(achAdapter, interbank, "ACH Transfer")

' =====

' EVENTS

' =====

Rel(transferDomain, eventPublisher, "TransferCompleted")

Rel(eventPublisher, kafka, "Publish Event")

Rel(transferDomain, auditPublisher, "Audit Event")

Rel(auditPublisher, kafka, "Publish Event")

LAYOUT_WITH_LEGEND()

@enduml

@startuml 04_oauth_sequence

title OAuth2 Authorization Code + PKCE + MFA Authentication Flow

actor User

participant "SPA / Mobile App" as APP

participant "Identity Provider (OIDC)" as IDP

participant "API Gateway" as API

participant "JWKS Endpoint" as JWKS

participant "Microservice" as MS

== Authentication Request ==

User -> APP : Login

APP -> IDP : Authorization Request\n(PKCE Challenge)

IDP -> User : Login Screen

User -> IDP : Username + Password

== MFA Challenge ==

IDP -> User : MFA Required

alt Mobile Biometric

User -> IDP : Face ID / Touch ID

else OTP

User -> IDP : One Time Password

else Push Approval

User -> IDP : Push Confirmation

else Passkey (FIDO2)

User -> IDP : Passkey Authentication

end

== Authorization Code ==

IDP --> APP : Authorization Code

APP -> IDP : Authorization Code\n+ PKCE Verifier

IDP --> APP : Access Token (JWT)\nRefresh Token

== API Access ==

APP -> API : HTTPS + JWT Access Token

API -> JWKS : Get Public Keys

JWKS --> API : Public Keys

API -> API : Validate Signature

API -> API : Validate Claims\n(exp, aud, iss)

API -> API : Validate Scopes

API -> MS : Authorized Request

MS --> API : Business Response

API --> APP : Response

note right of IDP

Recommended OAuth2 Flow:

- Authorization Code
- PKCE Mandatory
- OpenID Connect

Recommended MFA:

- Mobile Biometrics
- OTP
- Push Approval
- Passkeys (FIDO2)

end note

note right of API

Security Controls:

- JWT Validation
- Rate Limiting
- Throttling
- WAF Protection

end note

@enduml

@startuml 05_onboarding_sequence

title Mobile Digital Onboarding Flow

actor Customer

participant "Flutter App" as APP

participant "API Gateway" as API

participant "Onboarding Service" as ONB

participant "OCR Service" as OCR

participant "Biometric Provider" as BIO

participant "KYC/AML Service" as KYC

participant "Identity Provider" as IDP

participant "Audit Service" as AUDIT

participant "Kafka Event Bus" as KAFKA

participant "Notification Service" as NOTIF

== Document Capture ==

Customer -> APP : Capture identity document

APP -> API : document Upload

API -> ONB : Start Onboarding

ONB -> OCR : Extract Identity Data

OCR --> ONB : Identity Data

ONB -> AUDIT : Audit(Document Captured)

== Selfie Capture ==

Customer -> APP : Take a selfie

APP -> API : Upload selfie

API -> ONB : Selfie

ONB -> AUDIT : Audit(Selfie Captured)

== Biometric Validation ==

ONB -> BIO : Liveness Detection

BIO --> ONB : Liveness Passed

ONB -> BIO : Face Match

BIO --> ONB : Match Score

ONB -> AUDIT : Audit(Biometric Validation)

alt Match Score < Threshold

ONB -> AUDIT : Audit(Onboarding Rejected)

ONB -> KAFKA : OnboardingRejected

ONB --> APP : Onboarding Rejected

else Match Score >= Threshold

== Compliance Validation ==

ONB -> KYC : AML Screening

KYC --> ONB : Passed

ONB -> KYC : PEP Screening

KYC --> ONB : Passed

ONB -> KYC : Sanctions Check

KYC --> ONB : Passed

ONB -> AUDIT : Audit(KYC AML Approved)

== Identity Provisioning ==

ONB -> IDP : Create User

IDP --> ONB : User Created

ONB -> AUDIT : Audit(User Created)

== MFA Enrollment ==

ONB -> IDP : Enroll MFA

IDP --> ONB : MFA Enabled

ONB -> AUDIT : Audit(MFA Enabled)

== Event Publication ==

ONB -> KAFKA : CustomerOnboarded

ONB -> KAFKA : AuditEvent

== Notifications ==

KAFKA -> NOTIF : Consume CustomerOnboarded

NOTIF -> APP : Welcome Notification

== Audit Trail ==

KAFKA -> AUDIT : Consume AuditEvent

AUDIT -> AUDIT : Persist Audit Record

== Success ==

ONB --> APP : Onboarding Success

end

@enduml

@startuml 06_event_driven

title Event Driven Architecture (Enterprise Banking)

top to bottom direction

skinparam linetype ortho

rectangle "Transfer Service" as TS

queue "Kafka Event Bus" as KAFKA

database "Schema Registry" as SCHEMA

queue "Retry Topic" as RETRY

queue "Notification DLQ" as DLQ

rectangle "Audit Service" as AUDIT

database "Audit DB" as AUDITDB

rectangle "Notification Service" as NOTIF

database "Notification Preferences" as PREFS

rectangle "Fraud Monitoring Service" as FRAUD

rectangle "Analytics Service" as ANALYTICS

cloud "Email Provider" as EMAIL

cloud "SMS Provider" as SMS

cloud "Push Provider" as PUSH

node "Observability Stack\nPrometheus + Grafana + OpenSearch" as OBS

' =====

' EVENT PUBLICATION

' =====

TS --> KAFKA : TransferInitiated

TS --> KAFKA : TransferCompleted

TS --> KAFKA : TransferRejected

KAFKA --> SCHEMA : Validate Event Schema

note right of SCHEMA

Schema Versioning

Backward Compatibility

Contract Governance

end note

' =====

' AUDIT

' =====

KAFKA --> AUDIT : Consume Business Events

AUDIT --> AUDITDB : Store Audit Record

' =====

' NOTIFICATIONS

' =====

KAFKA --> NOTIF : Consume Events

NOTIF --> PREFS : Resolve Preferences

PREFS --> NOTIF : Channels Enabled

NOTIF --> EMAIL : Email Notification

NOTIF --> SMS : SMS Notification

NOTIF --> PUSH : Push Notification

NOTIF --> RETRY : Failed Delivery

RETRY --> NOTIF : Retry Processing

RETRY --> DLQ : Max Retries Reached

' =====
' FRAUD MONITORING
' =====

KAFKA --> FRAUD : Consume Events

FRAUD --> FRAUD : AML Analysis

FRAUD --> FRAUD : Behavioral Analysis

' =====
' ANALYTICS
' =====

KAFKA --> ANALYTICS : Consume Events

ANALYTICS --> ANALYTICS : KPIs / Reporting

' =====
' OBSERVABILITY
' =====

TS --> OBS : Metrics / Logs

KAFKA --> OBS : Broker Metrics

SCHEMA --> OBS : Schema Metrics

RETRY --> OBS : Retry Metrics

DLQ --> OBS : Failed Events Metrics

AUDIT --> OBS : Metrics / Logs

NOTIF --> OBS : Metrics / Logs

FRAUD --> OBS : Metrics / Logs

ANALYTICS --> OBS : Metrics / Logs

note right of OBS

Monitoring:

- Consumer Lag
- Failed Deliveries
- DLQ Growth
- Topic Throughput
- Broker Health

Auto-Healing:

- Consumer Restart
- Pod Restart
- Alert Escalation

end note

@enduml

@startuml 07_aws_infrastructure

top to bottom direction

skinparam linetype ortho

!define AWSPuml ../aws-icons-for-plantuml/dist

!include AWSPuml/AWSCommon.puml

!include AWSPuml/NetworkingContentDelivery/Route53.puml

!include AWSPuml/NetworkingContentDelivery/CloudFront.puml

!include AWSPuml/SecurityIdentityCompliance/WAF.puml

!include AWSPuml/NetworkingContentDelivery/ElasticLoadBalancing.puml

!include AWSPuml/Containers/ElasticKubernetesService.puml

!include AWSPuml/Containers/ElasticContainerRegistry.puml

!include AWSPuml/Database/AuroraAmazonRDSInstance.puml

!include AWSPuml/Database/ElastiCache.puml

!include AWSPuml/Analytics/ManagedStreamingforApacheKafka.puml

!include AWSPuml/SecurityIdentityCompliance/SecretsManager.puml

!include AWSPuml/SecurityIdentityCompliance/KeyManagementService.puml

!include AWSPuml/ManagementGovernance/CloudWatch.puml

!include AWSPuml/Analytics/OpenSearchService.puml

title BP Internet Banking - AWS Production Architecture

cloud "Internet Users" as users

Route53(route53,"Amazon Route53","Global DNS")

CloudFront(cloudfront,"Amazon CloudFront","CDN")

WAF(waf,"AWS WAF + Shield Advanced","DDoS & OWASP Protection")

ElasticLoadBalancing(alb,"Application Load Balancer","HTTPS Entry Point")

users --> route53

route53 --> cloudfront

cloudfront --> waf

waf --> alb

rectangle "AWS VPC" {

rectangle "Public Subnets" {

[ALB Endpoint]

component nat [

NAT Gateway

]

}

rectangle "Private Subnets" {

component eksControlPlane [

Amazon EKS Control Plane

]

rectangle "Availability Zone A" {

ElasticKubernetesService(eksA, "EKS Worker Nodes A", "Microservices")

}

rectangle "Availability Zone B" {

ElasticKubernetesService(eksB, "EKS Worker Nodes B", "Microservices")

}

```
rectangle "Availability Zone C" {
```

```
    ElasticKubernetesService(eksC, "EKS Worker Nodes C", "Microservices")
```

```
}
```

```
rectangle "Platform Services" {
```

```
    component istio [
```

```
        Istio Service Mesh
```

```
        mTLS + Retry + Circuit Breaker
```

```
    ]
```

```
    component hpa [
```

```
        Horizontal Pod Autoscaler
```

```
    ]
```

```
    component autoscaler [
```

```
        Cluster Autoscaler
```

```
    ]
```

```
    component ingress [
```

```
        NGINX Ingress Controller
```

```
    ]
```

```
}
```

```
rectangle "Business Services" {
```

```
    component api [
```

```
        API Gateway
```

```
    ]
```


component customer [
Customer Service
]

component account [
Account Service
]

component transaction [
Transaction Service
]

component transfer [
Transfer Service
]

component payment [
Payment Service
]

component onboarding [
Onboarding Service
]

component notification [
Notification Service
]

component audit [
Audit Service
]

component fraud [
Fraud Service

]

}

component cognito [

Amazon Cognito

OAuth2 / OIDC

MFA

]

AuroraAmazonRDSInstance(aurora, "Aurora PostgreSQL Global Database", "Multi-AZ")

ElastiCache(redis, "ElastiCache Redis", "Multi-AZ + Global Datastore Ready")

ManagedStreamingforApacheKafka(msk, "Amazon MSK", "Kafka Event Bus")

SecretsManager(secrets, "Secrets Manager", "Credentials")

KeyManagementService(kms, "AWS KMS", "Encryption Keys")

CloudWatch(cloudwatch, "CloudWatch", "Metrics & Alarms")

OpenSearchService(opensearch, "OpenSearch", "Centralized Logs")

component tracing [

AWS X-Ray / OpenTelemetry

]

}

}

ElasticContainerRegistry(ecr, "Amazon ECR", "Container Registry")

storage "AWS Backup Vault\nCross Region Backup" as backup

cloud "External Services" {

[Core Banking]

[Customer Information System]

[Biometric Provider]

[KYC / AML Provider]

[ACH Network]

[Email Provider]

[SMS Provider]

[Push Provider]

}

alb --> ingress

ingress --> istio

istio --> api

api --> customer

api --> account

api --> transaction

api --> transfer

api --> payment

api --> onboarding

api --> cognito

customer --> aurora

account --> aurora

transaction --> aurora

transfer --> aurora

payment --> aurora

onboarding --> aurora

customer --> redis

transfer --> redis

transfer --> msk

payment --> msk

onboarding --> msk

notification --> msk

audit --> msk

fraud --> msk

customer --> secrets

account --> secrets

transaction --> secrets

transfer --> secrets

payment --> secrets

secrets --> kms

customer --> cloudwatch

account --> cloudwatch

transaction --> cloudwatch

transfer --> cloudwatch

payment --> cloudwatch

notification --> cloudwatch

audit --> cloudwatch

fraud --> cloudwatch

customer --> tracing

account --> tracing

transaction --> tracing

transfer --> tracing

payment --> tracing

customer --> opensearch

account --> opensearch

transaction --> opensearch

transfer --> opensearch

payment --> opensearch

ecr --> eksA

ecr --> eksB

ecr --> eksC

aurora --> backup

hpa --> eksA

hpa --> eksB

hpa --> eksC

autoscaler --> eksA

autoscaler --> eksB

autoscaler --> eksC

@enduml

@startuml 08_ha_dr

title BP Internet Banking - High Availability, Disaster Recovery & Auto-Healing

top to bottom direction

skinparam linetype ortho

legend right

|= Estrategia | = Implementación |

| Alta Disponibilidad | Multi-AZ |

| Recuperación ante Desastres | Multi-Region |

| Auto-Healing | Kubernetes HPA + Cluster Autoscaler |

| Base de Datos | Aurora Global Database |

| Cache Distribuido | Redis Global Datastore |

| Mensajería | Amazon MSK Replication |

| RPO | <= 5 minutos |

| RTO | <= 30 minutos |

endlegend

cloud "Clientes" as users

rectangle "Route53 Health Checks\n+ Failover Routing" as route53

cloud "AWS Region Primaria" as primary {

frame "Availability Zone A" as az1 {

node "EKS Node Group A" {

component "API Gateway Pods"

component "Customer Service Pods"

component "Transfer Service Pods"

}

}

frame "Availability Zone B" as az2 {

node "EKS Node Group B" {

component "API Gateway Pods"

component "Account Service Pods"

component "Payment Service Pods"

}

}

frame "Availability Zone C" as az3 {

node "EKS Node Group C" {

component "Notification Service Pods"

component "Audit Service Pods"

component "Onboarding Service Pods"

}

}

database "Aurora PostgreSQL\nGlobal Database\nMulti-AZ" as aurora

collections "Redis Cluster\nPrimary + Replica" as redis

queue "Amazon MSK\nKafka Cluster\nMulti-AZ" as kafka

component "Horizontal Pod Autoscaler" as hpa

component "Cluster Autoscaler" as autoscaler

component "Liveness Probes" as liveness

component "Readiness Probes" as readiness

component "CloudWatch\nPrometheus\nGrafana\nOpenSearch" as monitoring

}

cloud "AWS DR Region" as dr {

database "Aurora Global Database\nCross Region Replica" as drAurora

collections "Redis Global Datastore\nReplica" as drRedis

queue "MSK Replicated Cluster" as drKafka

storage "Cross Region Backups" as backups

node "Warm Standby EKS Cluster" as drEKS

component "CloudWatch DR" as drMonitoring

}

users --> route53

route53 --> primary : Primary Route

route53 --> dr : Automatic Failover

'=====

' DATA REPLICATION

'=====

aurora --> drAurora : Continuous Replication

redis --> drRedis : Global Datastore Replication

kafka --> drKafka : Topic Replication

aurora --> backups : Automated Backups

'=====

' AUTO HEALING

'=====

hpa --> az1

hpa --> az2

hpa --> az3

autoscaler --> az1

autoscaler --> az2

autoscaler --> az3

liveness --> az1

liveness --> az2

liveness --> az3

readiness --> az1

readiness --> az2

readiness --> az3

'=====

' MONITORING

'=====

az1 --> monitoring

az2 --> monitoring

az3 --> monitoring

drEKS --> drMonitoring

drAurora --> drMonitoring

drKafka --> drMonitoring

'=====

' NOTES

'=====

note right of hpa

Auto-Healing:

- Escala pods automáticamente
- Basado en CPU
- Basado en memoria
- Basado en latencia

end note

note right of autoscaler

Escala nodos EKS
según demanda

end note

note right of liveness

Reinicia automáticamente
contenedores no saludables

end note

note right of readiness

Retira pods defectuosos
del balanceador

end note

note bottom of dr

Warm Standby:

- EKS preaprovisionado
- Aurora replicada

- Redis replicado

- Kafka replicado

Activación automática

ante desastre regional

end note

@enduml